

本节讲解的例题及其囊括的知识点，如表4-8所示。

表4-8 离线算法例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 4-36	NC 19919	动态逆序对	基于时间分治；三维偏序	陈锋
例 4-37	HDU 5552	Bus Routes	基于时间分治优化 DP；NTT	陈锋
例 4-38	SPOJ METEORS	Meteors	整体二分	陈锋
例 4-39	CodeChef AMCOINS	Coins	整体二分；树上差分	陈锋
例 4-40	SPOJ DQUERY	D-query	莫队	陈锋
例 4-41	NC 202003	数颜色	带修改莫队	陈锋

4.7 kd-Tree

kd-Tree是一种在 k 维欧几里得空间中组织点的数据结构，可以应用在多种场合，如范围搜寻及最邻近搜索等，有时也可以用来替代树套树。

例4-42 【kd-Tree；最近邻点查询】寻找酒店（Finding Hotels, ACM/ICPC青岛2016, HDU 5992）

给出 N ($N \leq 200\,000$) 个酒店，每个酒店用平面上的一个整点表示，并且有不同的价格。给出 M ($M \leq 20\,000$) 次询问，每次询问 (x, y, c) ，其中 $1 \leq x, y, c \leq N$ ，求出距离 (x, y) 最近的价格不超过 c 的酒店。

【分析】

使用kd-Tree查找与指定点最近的点即可，但是查找时要求候选点 p 对应的价格满足所求的条件。在实现上还应注意以下几个细节。

- 因为kd-Tree在建立过程中会改变点之间的顺序，所以要在普通的Point中增加一个id，记录此点的原始序号。

- ❑ Build过程中，不是交替选择划分的维度，而是看点集中于哪个维度时最大值和最小值差更大，则选择此维度。
- ❑ 因为是整数点，为了不引入浮点误差，查找过程中记录最短距离的平方。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
typedef long long LL;
using namespace std;
const int NN = 2E5 + 8;
struct Point {
    LL x, y;
    int c, id;
} Ps[NN];
istream& operator>>(istream& is, Point& p) { return is >> p.x >> p.y >> p.c; }
bool cmpx(const Point& p1, const Point& p2) { return p1.x < p2.x; }
bool cmpy(const Point& p1, const Point& p2) { return p1.y < p2.y; }
```

```

LL dist(const Point& a, const Point& b) {
    return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
}
bool Div[NN]; // 每一层的划分方式
void build(int l, int r) {
    if (l > r) return;
    int m = (l + r) / 2;
    Point *pl = Ps + l, *pr = Ps + r + 1;
    pair<Point*, Point*> px = minmax_element(pl, pr, cmpx),
        py = minmax_element(pl, pr, cmpy);
    Div[m] = px.second->x - px.first->x >= py.second->y - py.first->y;
    nth_element(pl, Ps + m, pr, Div[m] ? cmpx : cmpy);
    build(l, m - 1), build(m + 1, r);
}
// Ps[l,r]中距离p最小的点->id, 最小距离min_d, 且要求Ps[id].c<p.c
void nearest(int l, int r, const Point& p, LL& min_d, int& id) {
    if (l > r) return;
    int m = (l + r) / 2;
    const Point& pm = Ps[m];
    LL d = dist(p, pm);
    if (pm.c <= p.c) {
        if (d < min_d) min_d = d, id = m;
        else if (d == min_d && pm.id < Ps[id].id) id = m;
    }
    d = Div[m] ? (p.x - pm.x) : (p.y - pm.y);
    if (d <= 0) {
        nearest(l, m - 1, p, min_d, id);
        if (d * d < min_d) nearest(m + 1, r, p, min_d, id);
    } else {
        nearest(m + 1, r, p, min_d, id);
        if (d * d < min_d) nearest(l, m - 1, p, min_d, id);
    }
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int id, n, m, T;
    cin >> T;
    while (T--) {
        cin >> n >> m;
        for (int i = 1; i <= n; i++) cin >> Ps[i], Ps[i].id = i;
        build(1, n);
        Point p;
        while (m--) {
            cin >> p;
            LL min_d = 1LL << 60;
            nearest(1, n, p, min_d, id);
            printf("%lld %lld %d\n", Ps[id].x, Ps[id].y, Ps[id].c);
        }
    }
}

```

```

    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.10 节例题 51
同类题目：Closest Points, CodeChef CLOSEST
*/

```

例4-43 【kd-Tree; 莫队】保持健康 (Keep Fit, UVa 12939)

给出编号为 $1 \sim N$ 的 N ($1 \leq N \leq 200\,000$) 个点，每个点的坐标 x, y 都满足 ($|x|, |y| \leq 10^8$)。给出一个固定的参数 d ($1 \leq d \leq 10^8$)，以及 q ($1 \leq q \leq 1000$) 个询问，每个询问给出两个整数 i, j ，要求计算编号 k 符合 $i \leq k \leq j$ 的所有点中，有多少对点满足两点之间的曼哈顿距离不大于 d 。

注意：题目允许先读取所有的输入请求，计算完成后再输出。也就是说，允许使用离线算法。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (int)(b); ++i)
using namespace std;
typedef long long LL;

const int NN = 200010, MM = 10010;
int N, M, D, NodeId[NN], root, cmp_dim, BLOCK;
struct Point { int x, y; } PS[NN];
struct Query {
    int l, r, id;
    bool operator<(const Query& q) const {
        if (l / BLOCK != q.l / BLOCK) return l / BLOCK < q.l / BLOCK;
        return r < q.r;
    }
} QS[MM];
LL Ans[MM];
inline bool inRange(int x, int l, int r) { return l <= x && x <= r; }
struct KDTree {
    int xy[2], xyMax[2], xyMin[2], CH[2], cnt, cntSum, fa;
    bool operator<(const KDTree& k) const { return xy[cmp_dim] < k.xy[cmp_dim]; }
    inline int query(int x1, int x2, int y1, int y2);
    inline void update();
    inline void init(int i);
} Tree[NN];

// 查询整棵树在[x1,x1],[y1,y2]中的节点个数
inline int KDTree::query(int x1, int x2, int y1, int y2) {

```

```

int k = 0;
if (xyMin[0]>x2 || xyMax[0]<x1 || xyMin[1]>y2 || xyMax[1]<y1 || 0 == cntSum)
    return 0; // 整棵树都不在[x1, x2], [y1, y2]中
if (x1 <= xyMin[0] && xyMax[0] <= x2 && y1 <= xyMin[1] && xyMax[1] <= y2)
    return cntSum; // 整棵树都在其中
if (inRange(xy[0], x1, x2) && inRange(xy[1], y1, y2))
    k += cnt; // 当前点在其中
_for(i, 0, 2) if (CH[i])
    k += Tree[CH[i]].query(x1, x2, y1, y2); // 左右结点查询
return k;
}

// 更新当前整棵树的x,y的Min,Max值
inline void KDTree::update() { // 更新整棵树的x,y坐标的Max, Min值
    _for(i, 0, 2) if (CH[i]) _for(j, 0, 2) {
        xyMax[j] = max(xyMax[j], Tree[CH[i]].xyMax[j]);
        xyMin[j] = min(xyMin[j], Tree[CH[i]].xyMin[j]);
    }
}

// 初始化KDTree结点
inline void KDTree::init(int i) { // 初始化结点信息
    NodeId[fa] = i; // 一开始fa记录的是TreeNodeId对应的PointId
    _for(j, 0, 2) xyMax[j] = xyMin[j] = xy[j]; // 两个维度坐标的最大值
    cnt = cntSum = 0; // 是否在莫队当前区间中, 树中在莫队当前区间中的点个数, 初始都是0
    CH[0] = CH[1] = 0; // 左右子树初始化
}

// 将Ps[l, r]构建成一棵树
int build(int l, int r, int dim, int fa) {
    int mid = (l + r) / 2; // 区间分成两半
    // 取出按照cmp_dim维度对l,r点进行比较的中间点, 并且将点mid放在中间
    cmp_dim = dim, nth_element(Tree + l + 1, Tree + mid + 1, Tree + r + 1);
    KDTree& n = Tree[mid]; // 本树根结点
    n.init(mid), NodeId[n.fa] = mid, n.fa = fa;
    if (l < mid) n.CH[0] = build(l, mid - 1, !dim, mid); // 递归构建左子树
    if (r > mid) n.CH[1] = build(mid + 1, r, !dim, mid); // 递归构建右子树
    n.update();
    return mid;
}

LL curAns;
inline void addPos(int i) { // 查找所有与Ps[i]距离小于等于D的点的个数
    curAns += Tree[root].query(PS[i].x-D, PS[i].x+D, PS[i].y-D, PS[i].y+D);
    int ti = NodeId[i]; // 点i对应的KDTree结点
    Tree[ti].cnt = 1; // 将点i记录下来, 并且更新其所有祖先的计数
    while (ti) Tree[ti].cntSum++, ti = Tree[ti].fa;
}

```

```

inline void delPos(int i) {
    int ti = NodeId[i];
    Tree[ti].cnt = 0;
    // 将点 i 从莫队区间中去除, 更新所有父结点的计数
    while (ti) Tree[ti].cntSum--, ti = Tree[ti].fa;
    // 去掉跟点 i 距离不大于 D 的点的个数
    curAns -= Tree[root].query(PS[i].x-D, PS[i].x+D, PS[i].y-D, PS[i].y+D);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (int t = 1, x, y; cin >> N >> D >> M; t++) {
        printf("Case %d:\n", t);
        BLOCK = (int)sqrt(N + 0.5);
        _rep(i, 1, N) {
            KDTree &nd = Tree[i];
            cin >> x >> y;
            nd.xy[0] = PS[i].x = x + y, nd.xy[1] = PS[i].y = x - y, Tree[i].fa = i;
        }
        root = build(1, N, 0, 0);
        _rep(i, 1, M) cin >> QS[i].l >> QS[i].r, QS[i].id = i;
        sort(QS + 1, QS + M + 1);
        int curL = 1, curR = 0;
        curAns = 0;
        _rep(i, 1, M) {
            // 维护莫队当前的区间
            while (curR < QS[i].r) addPos(++curR);
            while (curR > QS[i].r) delPos(curR--);
            while (curL < QS[i].l) delPos(curL++);
            while (curL > QS[i].l) addPos(--curL);
            Ans[QS[i].id] = curAns;
        }
        _rep(i, 1, M) printf("%lld\n", Ans[i]);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.10 节例题 52
同类题目: Generating Synergy, Ipsc 2015G
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点, 如表4-9所示。

表4-9 kd-Tree例题归纳